Implementation Plan

Overview

MediFlow is built in 5 phases over about 7 weeks, with up to 4 coding agents working in parallel and you supervising: briefing each agent, reviewing its pull request, and only opening the next phase when a gate passes. The order follows the dependencies: every module needs the foundation, and the coordinator needs every other module as a tool.

Roadmap

Four agents in parallel, five gates, about seven weeks

	Foundation Week 1	**Core modules** Weeks 2–3	**Patient + money** Weeks 4–5	**Agent + evals** Week 6	**Launch** Week 7
Agent 1	Repo, core, data	M1 Scribe	M4 Messenger	**M5 Coordinator**	Hardening + docs
Agent 2		M2 Knowledge	M6 Claims + NHIS	Evals + Results	Deploy + video
Agent 3		M3 Data + model	M3 Scoring		
Agent 4		UI shell + design	Chat + claim UI		
Gate	◆ Stack runs	◆ Note approved	◆ Each one works	◆ Ama end to end	◆ Public demo

Build roadmap · 5 phases, up to 4 agents in parallel

The week counts are an estimate that assumes you review each agent's work within a day; the gates, not the dates, decide when a phase ends.

Phases

Phase	Agent	Workstream	Brief to hand the agent	Depends on
0. Foundation	1	Repo, Docker Compose, `core` services (LLM layer, masking, audit, config), all migrations from the Backend Schema, seed patients, Module 1 transcript generator	TRD, Backend Schema, Module 1 § Synthetic data	—
1. Core modules	1	Module 1 Scribe: extraction, validation, unmasking, Note review screen	Module 1 doc, Design Brief	Phase 0
1. Core modules	2	Module 2: generate the 3 PDFs and 30 questions, ingestion, hybrid search, Ask guidelines, Check guideline API	Module 2 doc	Phase 0 (transcripts)
1. Core modules	3	Module 3: appointments generator, training script, model card	Module 3 doc	Phase 0 (patients)
1. Core modules	4	React app shell: sidebar, role switcher, demo banner, shared components from the Design Brief, Knowledge base and Demo controls screens	Design Brief, App Flow	Phase 0
2. Patient + money	1	Module 4 Messenger: daily job, templates, after-visit rewrite, intent agent, emergency screen	Module 4 doc	M1, M2, M3

Phase	Agent	Workstream	Brief to hand the agent	Depends on
2. Patient + money	2	Module 6: tariffs and claims generator, rule engine, LLM check, simulated NHIS service	Module 6 doc	M1, M2
2. Patient + money	3	Module 3 scoring: feature builder, SHAP, explanation, Schedule screen	Module 3 doc	M3 model
2. Patient + money	4	Patient chat, Outbox, Nurse inbox, Alerts, Claims queue and Claim detail screens	App Flow, Design Brief, Module 4 and 6 docs	Agents 1–2 APIs
3. Agent + evals	1	Module 5 Coordinator: tools wrapping M2–M4 and M6, agent loop, care plan screen with trace	Module 5 doc	All modules
3. Agent + evals	2	All eval scripts wired to the Results screen; end-to-end test of Ama's journey	Each module's KPI section	All modules
4. Launch	1	Fix gaps from evals, README, model card, ADRs, compliance note	PRD, all docs	Phase 3
4. Launch	2	Deploy to a host with a public URL; record the 3-minute demo	PRD § Product goal	Phase 3

Milestones and gates

Gate	Passes when	Target
G0 Stack runs	`docker compose up` starts all 4 services; migrations apply; 10 transcripts and seed patients load	End of week 1

Gate	Passes when	Target
G1 Note approved	Ama's note goes transcript → draft → guideline check → approved; 3 PDFs ingested; no-show model trained and beats the baseline	End of week 3
G2 Each one works	Reminders and chat work; a claim is checked and submitted to the simulated NHIS; appointments show risk bands	End of week 5
G3 Ama end to end	Approving Ama's note produces a care plan; approving it books, messages and claims; all eval scripts run	End of week 6
G4 Public demo	Live URL, README with results table, demo video	End of week 7

How to run the agents

- **One agent, one branch, one module.** Name branches `m1-scribe`, `m2-knowledge` and so on; each agent works only in its module's package plus its screens.
- **Every brief = this pack + the module doc.** Always attach the TRD, Backend Schema and Design Brief, so shared rules are never re-invented.
- **Shared code changes go through you.** If an agent needs a change in `core` or the schema, it proposes it in its PR description; you merge it first, then the others rebase.
- **Merge order in a phase:** backend before screens. Agent 4's screens merge after the API they call.
- **Your review checklist for every PR:** it runs with `docker compose up`; its eval script passes or explains the gap; no prompt is inlined in code; every LLM call goes through `core/llm.py`; masking is used; guardrail tests exist and pass.

Risks

Risk	Effect	Fallback
Two agents change the same shared file	Merge conflicts, broken builds	Shared changes only through you, merged first
Synthetic data doesn't line up across modules	Demo breaks at hand-offs	Generators run in order (M1 → M2 → M3 → M6) and read each other's output
LLM cost during evals	Unexpected bills	Run evals on demand, not on every push; log cost per run

Risk	Effect	Fallback
Coordinator is unreliable	Weak demo moment	`COORDINATOR_ENABLED=false` keeps the direct flow working